ESPECIFICAÇÃO TÉCNICA

RetroGames — Arquitetura e Stack

Versão 1.0 · Escopo: Módulo de Catálogo e Administração

1. Visão geral

1.1 Objetivo

Este documento especifica a arquitetura, a stack tecnológica, o modelo de dados, os contratos da API e os requisitos de segurança do sistema RetroGames, uma loja web de jogos retrô usados. O escopo desta versão é o Módulo de Catálogo e Administração.

1.2 Funcionalidades

A tabela a seguir delimita as funcionalidades contempladas nesta versão.

Funcionalidade
Catálogo público de jogos, com busca textual, filtros por plataforma e gênero e paginação
Consulta de detalhe de jogo
Manutenção de plataformas (inclusão, consulta, alteração e exclusão)
Manutenção de jogos, com associação a gêneros (relação N:N) e exclusão lógica
Consulta de gêneros e de condições de jogo (tabelas de domínio)
Autenticação e autorização do perfil Administrador

1.3 Visão de componentes

O sistema é composto por três componentes executados localmente: uma Single Page Application (SPA) em React, uma API REST em ASP.NET Core e um banco de dados relacional MySQL. A SPA comunica-se exclusivamente com a API por HTTP/JSON. A API acessa o banco de dados por meio do ADO.NET, com o provedor MySqlConnector.

Em ambiente de desenvolvimento, o servidor do Vite (porta 5173) encaminha as requisições com prefixo `/api` para a API (porta 5080). Front-end e API compartilham a mesma origem, o que dispensa a configuração de CORS e permite o uso de cookies SameSite=Strict.

2. Stack tecnológica

As versões abaixo são as de referência do projeto.

Camada	Tecnologia	Versão	Finalidade
Plataforma	.NET SDK	10	Runtime e compilador C#
API	ASP.NET Core Web API (controllers)	10	Roteamento HTTP, injeção de dependência, autenticação e autorização
Acesso a dados	ADO.NET (System.Data.Common)	nativo	Abstrações de conexão, comando, leitura e transação
Provedor ADO.NET	MySqlConnector (NuGet)	2.6.2	Implementação do ADO.NET para o MySQL
Banco de dados	MySQL	8	Persistência relacional
Validação	FluentValidation	12.1.0	Validação de formato dos modelos de entrada
Documentação da API	OpenAPI + Swagger UI	10.x	Especificação e teste dos endpoints (somente em desenvolvimento)
Criptografia de senha	PasswordHasher (ASP.NET Core Identity)	nativo	Hash de senha com salt (PBKDF2)
Testes	xUnit v3 + Moq	3.0.0 / 4.20.72	Testes unitários com objetos simulados (mocks)
Front-end	React + Vite	19 / 8	SPA e servidor de desenvolvimento
Interface	Bootstrap	5	Layout responsivo e componentes visuais
Roteamento no cliente	React Router	7	Navegação entre telas
Cliente HTTP	Axios	1.x	Requisições à API e envio do token CSRF
Ferramentas	Git, VS Code, Node.js 22 LTS	—	Versionamento e ambiente de desenvolvimento

3. Arquitetura

3.1 Estilo arquitetural

O back-end adota arquitetura em camadas. Cada camada depende apenas da camada imediatamente inferior:

Controller → Service → Repository → banco de dados.

É vedado ao controller acessar o repositório diretamente. Somente a camada Repository contém instruções SQL. A camada Service não depende de tipos HTTP (`HttpContext`, códigos de status). O controle de transações é responsabilidade exclusiva da camada Service. Há um Repository por tabela, com o nome dela sem o prefixo tb_ (tb_condicao_jogo → CondicaoJogoRepository); cada um retorna entidades de uma única tabela, sem junções nem subconsultas em outras tabelas; filtros, ordenação e paginação são aplicados no SQL. A composição de dados de várias tabelas é feita no Service, com LINQ, e resulta em um ServiceModel. Filtros que dependem de outra tabela, como o filtro por gênero, também são resolvidos no Service: a seleção é feita com LINQ e o resultado segue para o SQL como lista de identificadores, o que mantém a contagem e a paginação no banco.

3.2 Projetos da solução

A solução `RetroGames.sln` é composta por seis projetos .NET e um projeto front-end.

Projeto	Responsabilidade	Dependências
`RetroGames.Model`	Entidades de domínio, constantes e tipos compartilhados	—
`RetroGames.Repository`	Repositórios ADO.NET e agregador IRepositorySet	Model
`RetroGames.Services`	Regras de negócio, ServiceModels, catálogo de erros de negócio e BusinessRuleException	Model, Repository, WebApi.ApiModels
`RetroGames.WebApi.ApiModels`	Modelos de entrada e saída da API, envelope de resposta e ResponseCodes	—
`RetroGames.WebApi`	Controllers, validators, mappers, configuração e tratamento global de exceções	Todos os anteriores
`RetroGames.WebApi.UnitTests`	Testes unitários	WebApi, Services
`retrogames.web`	SPA React	API, via HTTP

3.3 Responsabilidades por camada

Cada componente do back-end possui responsabilidade única, conforme a tabela.

Componente	Responsabilidade	Restrição
`Controller`	Receber a requisição, converter ApiModel em entidade (mapper), acionar o service e devolver o envelope com o status HTTP	Não contém regra de negócio nem SQL
`Validator`	Validar formato dos dados de entrada: obrigatoriedade, tamanho e faixa de valores	Não acessa o banco de dados

`Service`	Aplicar as regras de negócio, controlar transações (TransactionScope), compor dados de várias tabelas com LINQ em ServiceModels e lançar BusinessRuleException quando uma regra é violada	Não depende de HTTP nem contém SQL
`Repository`	Abrir conexão, executar SQL parametrizado e mapear resultados, participando da transação aberta pelo Service; cada consulta lê uma única tabela, com filtros, ordenação e paginação no SQL; converte a violação de `UNIQUE` (erro 1062) em `RegistroDuplicadoException`	Não decide regras de negócio nem controla transações
`GlobalExceptionHandler`	Converter BusinessRuleException no status HTTP do erro do catálogo (400, 404 ou 409) e outras exceções em 500, sempre no formato do envelope	—

3.4 Fluxo de processamento de uma requisição

Exemplo: inclusão de plataforma (`POST /api/Plataforma`).

1. Na tela de plataformas, o formulário chama `plataformaApi.inserir`, em `servicos.js`, que envia `POST /api/Plataforma` pelo Axios.
2. A requisição chega com corpo JSON e o header `X-CSRF-TOKEN`.
3. O middleware de antiforgery valida o token CSRF. Em seguida, o FluentValidation valida o `InserirPlataformaApiModel`; em caso de erro, a API responde HTTP 400 com responseCode 3.
4. O `PlataformaController` converte o ApiModel na entidade Plataforma e invoca `IPlataformaService.InserirAsync`.
5. O `PlataformaService` verifica a unicidade do nome.
    a. Em caso de duplicidade, lança `BusinessRuleException`, convertida em HTTP 409 (responseCode 101).
6. O `IPlataformaRepository.InserirAsync` executa INSERT parametrizado e retorna o identificador gerado (`LastInsertedId`).
7. Se outra requisição gravar o mesmo nome entre a verificação e o `INSERT`, o banco recusa a gravação (erro 1062); o Repository lança `RegistroDuplicadoException` e o Service a converte no mesmo erro de negócio (HTTP 409, responseCode 101).
8. O controller responde HTTP 201 com o recurso criado no envelope.
9. No front-end, `plataformaApi.inserir` devolve o conteúdo de `data`; em caso de erro, a tela exibe `errorMessage`.

3.5 Decisões de arquitetura

A tabela registra o motivo de cada decisão e indica se ela é uma boa prática geral ou uma escolha didática deste projeto. As escolhas didáticas simplificam o aprendizado; a última coluna indica o que é comum em sistemas profissionais ou a limitação da escolha.

Decisão	Motivo	Tipo	Alternativa ou limitação
Controller sem SQL e sem regra de negócio	Mantém a camada HTTP enxuta e permite testar o controller com o service simulado.	Boa prática	—

Regras de negócio no Service	Concentra as regras em um único lugar, independente de HTTP e do banco, e testável com mocks.	Boa prática	—
SQL parametrizado somente no Repository	Impede SQL injection e isola o acesso a dados: uma mudança no banco afeta uma única camada.	Boa prática	ORM (Entity Framework) ou micro-ORM (Dapper), fora do escopo do curso.
ApiModels separados das entidades	O contrato da API não muda quando a tabela muda, e campos internos, como `SenhaHash`, não são expostos.	Boa prática	—
Transação aberta pelo Service	Só o Service conhece a operação completa; os repositórios participam da transação sem controlá-la.	Boa prática	Unit of Work com transação explícita na conexão.
Uma tabela por consulta, com composição em LINQ	Cada classe espelha uma tabela, e a combinação dos dados fica visível no código do Service.	Escolha didática	No mercado: `JOIN` no SQL, views ou ORM. Limitações: mais consultas por operação (em número fixo) e ordenação por coluna de outra tabela fora do SQL.
Nenhuma regra no banco (sem `CHECK`, trigger ou function)	As regras ficam em um só lugar, onde são testadas e explicadas.	Escolha didática	No mercado, `CHECK` e restrições no banco são comuns como segunda proteção.
`UNIQUE` no banco e conferência no Service	O Service devolve a mensagem de negócio; o `UNIQUE` garante a regra quando duas requisições chegam juntas.	Boa prática	—

4. Modelo de dados

4.1 Convenção de nomenclatura

Tabelas e colunas utilizam nomes em português, em minúsculas, com prefixo que identifica o tipo do elemento.

Prefixo	Significado	Exemplo
`tb_`	Tabela	`tb_jogo`
`co_seq_`	Chave primária gerada pelo banco (AUTO_INCREMENT)	`co_seq_jogo`
`co_`	Chave fixa de tabela de domínio ou chave estrangeira	`co_condicao_jogo`
`no_ / ds_`	Nome / descrição	`no_jogo, ds_email`
`nu_ / qt_ / vl_`	Número / quantidade / valor monetário DECIMAL(10,2)	`nu_ano_lancamento, qt_estoque, vl_preco`
`dt_`	Data e hora em UTC	`dt_cadastro`
`st_`	Indicador booleano de situação	`st_ativo`

4.2 Tabelas do módulo

O módulo utiliza as tabelas abaixo.

Tabela	Tipo	Descrição	Relacionamentos
`tb_condicao_jogo`	Domínio	Condição de conservação do jogo	1:N com tb_jogo
`tb_perfil`	Domínio	Perfis de acesso (Administrador, Cliente)	1:N com tb_usuario
`tb_plataforma`	Cadastro	Console ou plataforma do jogo	1:N com tb_jogo
`tb_genero`	Cadastro (somente leitura)	Gênero do jogo	N:N com tb_jogo
`tb_jogo`	Principal	Jogo à venda: nome, descrição, ano, preço, estoque e situação	N:1 com plataforma e condição
`tb_jogo_genero`	Associativa	Associação entre jogo e gênero (chave primária composta)	N:1 com tb_jogo e tb_genero
`tb_usuario`	Cadastro	Usuário do sistema, com e-mail único e hash de senha	N:1 com tb_perfil

Dicionário de dados: as colunas de cada tabela estão definidas abaixo. Somente `ds_jogo` aceita NULL; as demais colunas são obrigatórias.

Tabela	Colunas
`tb_condicao_jogo`	`co_condicao_jogo, ds_condicao_jogo`
`tb_perfil`	`co_perfil, ds_perfil`

`tb_plataforma`	`co_seq_plataforma, no_plataforma, no_fabricante, nu_ano_lancamento`
`tb_genero`	`co_seq_genero, no_genero`
`tb_jogo`	`co_seq_jogo, no_jogo, ds_jogo, nu_ano_lancamento, vl_preco, qt_estoque, st_ativo, dt_cadastro, co_plataforma, co_condicao_jogo`
`tb_jogo_genero`	`co_jogo, co_genero`
`tb_usuario`	`co_seq_usuario, no_usuario, ds_email, ds_senha_hash, st_ativo, dt_cadastro, co_perfil`

4.3 Restrições de integridade

- O banco de dados é responsável exclusivamente pela integridade estrutural: chaves primárias, chaves estrangeiras, tipos de dados e obrigatoriedade (`NOT NULL`) e unicidade (`UNIQUE`).
- É vedado o uso de triggers, functions e ações referenciais (`ON DELETE CASCADE`, `ON DELETE SET NULL`). As chaves estrangeiras mantêm o comportamento padrão do MySQL.
- Toda regra de negócio (unicidade, faixas de valores, exclusão lógica e manutenção de registros dependentes) é implementada na camada Service. Quando uma operação exige mais de um comando, como a gravação de um jogo e de suas associações em `tb_jogo_genero`, o Service abre a transação com `TransactionScope` e a conclui com `Complete()`. O Repository apenas executa os comandos SQL e é vedado a ele abrir, confirmar ou desfazer transações. A unicidade é conferida pelo Service e garantida também pelo `UNIQUE` do banco: se duas gravações simultâneas passarem pela conferência, o banco recusa a segunda (erro 1062), o Repository lança `RegistroDuplicadoException` e o Service a converte no erro de negócio correspondente.

5. Contrato da API

5.1 Convenções

As rotas seguem o padrão `api/[controller]`. Requisições e respostas utilizam JSON, com propriedades em camelCase. Identificadores de recurso são passados na rota (por exemplo, `/api/Jogo/{id}`) e filtros, na query string. Os códigos de status HTTP utilizados são:

Status	Significado	Uso no sistema
200 OK	Sucesso	Consultas e alterações
201 Created	Recurso criado	Inclusões (POST)
400 Bad Request	Requisição inválida	Falha de validação de formato ou regra de negócio violada
401 Unauthorized	Não autenticado	Ausência de sessão ou credenciais inválidas
403 Forbidden	Sem permissão	Usuário autenticado sem o perfil exigido
404 Not Found	Recurso inexistente	Registro não encontrado ou rota inexistente
409 Conflict	Conflito com dados gravados	Nome duplicado ou exclusão de registro com dependentes
500 Internal Server Error	Erro inesperado	Exceção não tratada

5.2 Endpoints

Níveis de acesso: Público (sem autenticação) e Administrador (sessão ativa com perfil Administrador).

Método	Rota	Acesso	Descrição
GET	`/api/Autenticacao/csrf`	Público	Emite o token antiforgery no cookie XSRF-TOKEN
POST	`/api/Autenticacao/login`	Público	Autentica o usuário e cria a sessão
POST	`/api/Autenticacao/logout`	Administrador	Encerra a sessão
GET	`/api/Autenticacao/eu`	Administrador	Retorna os dados do usuário autenticado
GET	`/api/Plataforma`	Público	Lista as plataformas
GET	`/api/Plataforma/{id}`	Público	Consulta uma plataforma
POST	`/api/Plataforma`	Administrador	Inclui uma plataforma
PUT	`/api/Plataforma/{id}`	Administrador	Altera uma plataforma
DELETE	`/api/Plataforma/{id}`	Administrador	Exclui uma plataforma sem jogos vinculados
GET	`/api/Genero`	Público	Lista os gêneros
GET	`/api/CondicaoJogo`	Público	Lista as condições de jogo
GET	`/api/Jogo`	Público	Lista paginada de jogos. Parâmetros: busca, idPlataforma, idGenero, pagina, tamanhoPagina, incluirInativos (somente Administrador)

GET	`/api/Jogo/{id}`	Público	Consulta um jogo, com seus gêneros
POST	`/api/Jogo`	Administrador	Inclui um jogo
PUT	`/api/Jogo/{id}`	Administrador	Altera um jogo
DELETE	`/api/Jogo/{id}`	Administrador	Desativa um jogo (exclusão lógica)

5.3 Envelope de resposta

Todas as respostas, de sucesso ou de erro, utilizam o envelope abaixo. Em falhas de validação (responseCode 3), data contém os erros indexados por campo. Os códigos são declarados em dois catálogos, sem repetição: ResponseCodes (projeto ApiModels), de 0 a 99, no formato (Code, DefaultMessage), com os códigos que nascem na própria API; e BusinessErrors (projeto Services), a partir de 100, no formato (Code, DefaultMessage, Status), com os erros de negócio e o status HTTP de cada um, indicado na tabela ao lado do código. O envelope devolve o código do erro de negócio lançado pelo Service. Em cada catálogo, um novo código recebe o próximo número livre. Os códigos do Módulo de Compras serão definidos na tarefa correspondente. A propriedade `data` é omitida nas respostas de erro, exceto na falha de validação. A seção 5.4 apresenta exemplos.

```
{ "responseCode": 0, "data": { ... }, "errorMessage": null }
```

Código	Constante	Mensagem padrão
0	`SUCESSO`	—
1	`ERRO`	Exceção não tratada; mensagem genérica definida no GlobalExceptionHandler
2	`MODEL_NULL`	O modelo veio nulo.
3	`MODEL_STATE_INVALIDO`	Há um erro com o modelo enviado.
4	`NAO_AUTENTICADO`	Faça login para continuar.
5	`ACESSO_NEGADO`	Você não tem permissão para esta operação.
6	`CREDENCIAIS_INVALIDAS`	E-mail ou senha inválidos.
100 · 404	`PLATAFORMA_NAO_ENCONTRADA`	Plataforma não encontrada.
101 · 409	`PLATAFORMA_NOME_DUPLICADO`	Já existe uma plataforma com este nome.
102 · 409	`PLATAFORMA_POSSUI_JOGOS`	Não é possível excluir: existem jogos desta plataforma.
103 · 404	`JOGO_NAO_ENCONTRADO`	Jogo não encontrado.
104 · 404	`CONDICAO_NAO_ENCONTRADA`	Condição de jogo não encontrada.
105 · 404	`GENERO_NAO_ENCONTRADO`	Gênero não encontrado.

5.4 Exemplos de requisição e resposta

Inclusão de plataforma (`POST /api/Plataforma`). As mensagens são as definidas nos validators, em ResponseCodes e em BusinessErrors.

Requisição

```
{ "nome": "Super Nintendo", "fabricante": "Nintendo", "anoLancamento": 1990 }
```

HTTP 201: inclusão realizada

```
{ "responseCode": 0, "data": { "id": 7, "nome": "Super Nintendo", "fabricante":
"Nintendo", "anoLancamento": 1990 }, "errorMessage": null }
```

HTTP 400: falha de validação (nome vazio e ano fora da faixa)

```
{ "responseCode": 3, "data": { "nome": ["Informe o nome."], "anoLancamento": ["Ano de
lançamento inválido."] }, "errorMessage": "Há um erro com o modelo enviado." }
```

HTTP 409: violação da RN01 (nome já cadastrado)

```
{ "responseCode": 101, "errorMessage": "Já existe uma plataforma com este nome." }
```

6. Requisitos de segurança

Os requisitos abaixo são obrigatórios em todas as funcionalidades do módulo.

ID	Requisito
RS01	A sessão deve ser mantida em cookie HttpOnly e SameSite=Strict, com expiração deslizante de 2 horas. Nenhum token de sessão pode ser armazenado em localStorage ou sessionStorage.
RS02	Requisições POST, PUT e DELETE devem conter o header X-CSRF-TOKEN, validado globalmente pelo filtro AutoValidateAntiforgeryToken. O token é obtido em GET /api/Autenticacao/csrf e renovado após login e logout.
RS03	Endpoints de manutenção devem exigir a política de autorização SomenteAdministrador. A proteção de rotas no front-end é apenas de usabilidade; o controle efetivo é feito pela API (401/403).
RS04	Senhas devem ser armazenadas exclusivamente como hash gerado pelo PasswordHasher. É vedado registrar senhas em log.
RS05	Em desenvolvimento, a cadeia de conexão e as credenciais do administrador inicial ficam no arquivo .env, na raiz do repositório, listado no .gitignore e carregado pela API com o pacote DotNetEnv; o repositório versiona apenas o modelo .env.example, sem senhas. Os demais arquivos appsettings são versionados e não contêm senhas. Em produção, esses valores vêm de variáveis de ambiente. É vedado versionar credenciais.
RS06	Todo valor recebido do cliente deve ser enviado ao banco como parâmetro de comando (@parametro). É vedada a concatenação de valores em instruções SQL.

7. Regras de negócio

Regras de negócio são implementadas na camada Service, exceto quando indicado. A violação de uma regra resulta em BusinessRuleException com o erro correspondente de BusinessErrors. Cada erro do catálogo traz o seu status HTTP: 400 para regra violada, 404 quando o registro não existe e 409 quando a operação conflita com dados já gravados.

ID	Regra	Implementação
RN01	Unicidade de plataforma. Não pode haver duas plataformas com o mesmo nome. Violação: HTTP 409, código 101.	PlataformaService
RN02	Integridade de plataforma. É vedada a exclusão de plataforma com jogos vinculados. Violação: HTTP 409, código 102.	PlataformaService
RN03	Exclusão lógica de jogos. A exclusão de um jogo deve atribuir st_ativo = 0, preservando o registro e seus relacionamentos.	JogoService e JogoRepository
RN04	Visibilidade do catálogo. A consulta pública retorna apenas jogos ativos. Jogos inativos são listados somente para o Administrador, mediante o parâmetro incluirInativos.	JogoService
RN05	Referências válidas. Todo jogo deve referenciar plataforma, condição e ao menos um gênero existentes. Violação: HTTP 404, códigos 100, 104 e 105.	JogoValidator e JogoService
RN06	Valores de jogo. O preço deve ser maior que zero e o estoque, maior ou igual a zero. O ano de lançamento deve estar entre 1970 e 2100.	JogoValidator e JogoService

7.1 Rastreabilidade

A tabela relaciona cada funcionalidade da seção 1.2 aos endpoints, às regras, às classes que a implementam e aos testes unitários que a verificam. O traço indica que ainda não há teste específico.

Funcionalidade	Endpoints	Regras	Classes principais	Testes
Catálogo público de jogos	GET /api/Jogo	RN04	JogoController, JogoService, JogoRepository	JogoServiceTests
Detalhe de jogo	GET /api/Jogo/{id}	RN04	JogoController, JogoService	—
Manutenção de plataformas	GET, POST, PUT e DELETE /api/Plataforma	RN01, RN02	PlataformaController, PlataformaService, PlataformaRepository e validators de plataforma	PlataformaControllerTests, PlataformaServiceTests, PlataformaValidatorsTests
Manutenção de jogos	POST, PUT e DELETE /api/Jogo	RN03, RN05, RN06	JogoController, JogoService, JogoRepository, JogoGeneroRepository e validators de jogo	JogoValidatorsTests
Gêneros e condições de jogo	GET /api/Genero, GET /api/CondicaoJogo	—	GeneroController, CondicaoJogoController	—
Autenticação e autorização	/api/Autenticacao (csrf, login, logout, eu)	RS01 a RS06	AutenticacaoController, AutenticacaoService, UsuarioRepository	AutenticacaoValidatorsTests

Todas	—	Envelope (5.3)	GlobalExceptionHandler, FluentValidationFilter	GlobalExceptionHandlerTests

8. Front-end

O front-end é uma SPA em React 19, construída com Vite e estilizada com Bootstrap 5. Cada tela é implementada em um componente de página próprio, sem componentes genéricos de tabela ou formulário.

8.1 Inicialização e rotas

Arquivos localizados na raiz de `src/`.

Arquivo	Responsabilidade
`main.jsx`	Ponto de entrada. Importa o CSS e o JavaScript do Bootstrap e monta a árvore BrowserRouter → AutenticacaoProvider → App
`App.jsx`	Declaração das rotas (React Router), agrupadas em públicas e restritas ao Administrador
`index.css`	Ajustes visuais complementares ao Bootstrap
`lib/formato.js`	Funções de formatação: formatarPreco (moeda BRL, pt-BR) e formatarDataHora

8.2 Camada de acesso à API (api/)

Os componentes não utilizam o Axios diretamente. Toda chamada à API é feita por meio de **`servicos.js`**.

Arquivo	Responsabilidade
`api.js`	Instância do Axios com baseURL /api e envio de cookies. Interceptor de requisição: em POST, PUT e DELETE, lê o cookie XSRF-TOKEN e envia o header X-CSRF-TOKEN. Interceptor de resposta: se o token for rejeitado, renova-o e repete a requisição uma única vez; em HTTP 401, emite o evento sessao:expirada
`servicos.js`	Uma função por endpoint, agrupadas em autenticacaoApi, plataformaApi, generoApi, condicaoJogoApi e jogoApi. Cada função retorna o campo data do envelope
`erros.js`	obterMensagemErro (extrai errorMessage) e obterErrosDosCampos (extrai os erros por campo quando responseCode = 3)

8.3 Estado global (contexts/ e hooks/)

O estado de autenticação é compartilhado por meio da Context API do React.

Arquivo	Estado mantido	Interface exposta
`AutenticacaoProvider.jsx`	Usuário autenticado, obtido em GET /api/Autenticacao/eu na inicialização. O estado é limpo ao receber o evento sessao:expirada	usuario, carregando, estaAutenticado, ehAdministrador, entrar, sair
`autenticacaoContext.js`	Contexto criado com createContext	—
`useAutenticacao.js`	Hook de acesso ao contexto de autenticação	Utilizado por páginas e componentes

8.4 Componentes reutilizáveis (components/)

Componentes de interface utilizados por mais de uma página.

Componente	Props	Responsabilidade
`Layout`	—	Barra de navegação (Catálogo, menu Administração, Entrar e Sair) e <Outlet /> para renderização da página corrente
`CardJogo`	`jogo`	Cartão do catálogo: plataforma, nome, ano, condição, gêneros, preço, indicador de esgotado e link para o detalhe
`Paginacao`	`pagina, totalPaginas, aoMudar`	Controles de paginação; não é renderizado quando há uma única página
`MensagensEstado`	`Carregando (texto), MensagemErro (mensagem, aoTentarNovamente), EstadoVazio (titulo, children)`	Componentes de estado de carregamento, de erro com nova tentativa e de lista vazia
`RotaProtegida`	`somenteAdministrador`	Restringe rotas no cliente. Sem sessão, redireciona para /login?voltarPara=…; sem o perfil exigido, redireciona para o catálogo

8.5 Páginas (pages/)

Cada página corresponde a uma rota da aplicação.

Página	Rota	Acesso	Responsabilidade
`Catalogo`	/	Público	Lista paginada de jogos com busca e filtros por plataforma e gênero, refletidos na query string
`DetalheJogo`	`/jogos/:id`	Público	Exibe os dados completos do jogo
`Login`	`/login`	Público	Autenticação por e-mail e senha, com retorno à rota de origem
`Jogos`	`/admin/jogos`	Administrador	Lista paginada com busca, incluindo inativos; ações de edição e desativação
`FormularioJogo`	`/admin/jogos/novo` e `/admin/jogos/:id/editar`	Administrador	Inclusão e alteração de jogo: plataforma, condição, gêneros (N:N), preço e estoque, com erros por campo
`Plataforma`	`/admin/plataformas`	Administrador	Manutenção de plataformas: listagem, inclusão, alteração e exclusão
`NaoEncontrada`	*	Público	Resposta para rotas inexistentes (404)

9. Testes

- **Tipo:** testes unitários com xUnit v3 e Moq. Testes de integração não fazem parte do escopo.
- **Alvos:** controllers, com o service substituído por mock; services, com o repositório substituído por mock; `GlobalExceptionHandler`. Também são testados os validators, com casos válidos e inválidos. A seção 7.1 relaciona os testes às funcionalidades e às regras.
- **Padrão:** Arrange, Act, Assert (AAA), com um único comportamento verificado por teste.
- **Execução:** `dotnet test` na raiz da solução. Todos os testes devem passar antes de cada commit na branch principal.

10. Execução local

Pré-requisitos: .NET SDK 10, Node.js 22 LTS e MySQL 8 em execução.

1. Criar o banco de dados e deixar executando.
2. Copiar .env.example para .env, na raiz do repositório, e preencher ConnectionStrings__Default, AdminInicial__Email e AdminInicial__Senha.
3. Executar `dotnet run` no projeto `RetroGames.WebApi`. A API responde na porta 5080 e o Swagger UI fica disponível em `/swagger`.
4. Executar `npm install`.
5. Execute `npm run dev` no diretório `retrogames.web`. A aplicação responde na porta 5173.
6. Executar `dotnet test` na raiz da solução para validar os testes unitários.